

Computer Architecture

Warning: Do not fill the personal information or look at the contents of the exam before instructed by the invigilators. Violations will be treated as academic misconduct.

Personal Information

Student ID: _____

Name: _____

Family Name: _____

Signature: _____

Exam Information:

Lecturer: Yuval Yarom

Exam Time: 03.02.2025 16:30

Semester: Winter Semester 2024/25

Exam Length: 120 minutes

Instructions:

- Fill your personal information before starting the exam.
- Write all answers within the designated boxes on the exam paper
- If you need more space, use the extra blank pages at the end of the exam
- Examination material must not be removed from the examination room
- All answers must be in English

Materials

- One A4 reference sheet – must be handed in with exam

Marking:

Question Mark	1	2	3	4	5	6	7
Question Mark	8	9	10	11	12	13	14
Question Mark	15	16	17	18	19	20	

Exam: _____ /100 + Bonus: _____ = Total:

Question 1**4 marks**

I designed a new algorithm that can break modern encryption within 20 hours. Of these, one hour is spent on set up, 90 minutes are for collecting results, and the rest is for number crunching. Google is super excited about my algorithm, and is offering that I use 35 of their computers to do the number crunching instead of using my one computer. Assuming that the number crunching phase is completely parallelizable, what speed up should I expect from using the Google servers?

Question 2**4 marks**

Represent the number -100 (minus one hundred) as an 8-bit binary number using:

Sign-magnitude:

Ones' complement:

Two's complement:

Please go on to the next page...

Question 3**4 marks**

Identify three main differences between RISC and CISC?

Question 4**4 marks**

I bought a new memory with timing parameters **DDR4-3000 CL 18-16-16**. What is the expected memory latency?

Please go on to the next page...

Question 5**4 marks**

What are **write through** and **write back**? What are the advantages and disadvantages of each?

Question 6**4 marks**

What is **forwarding** in a pipeline? Why is it used?

Please go on to the next page...

Question 7**4 marks**

Mark which hazard type affects which pipeline design.

	RAW	WAR	WAW
In-order:	✓		
Out-of-Order without register renaming			
Out-of-Order with register renaming			

Question 8**4 marks**

What is the access time for a hard disk that can position the head on the right track within 8 ms and rotates at 5000 RPM?

Question 9

4 marks

What is the main purpose of a Programmable Interrupt Controller (PIC)? How does it achieve its aims?

Please go on to the next page...

Question 10

6 marks

In this question we use Hamming code to encode and decode bit strings. In both parts, bit numbers grow from right to left, with the least significant bit being number 1. For convenience, we insert spaces between every group of four bits. These spaces are **not** part of the bit stream.

- (a) Encode 100 1011 0110

[illegible]

- (b) Identify the error bit and decode: 010 0001 1111 0101

--

Please write the answers in the boxes above. The tables below can be used as scratch pads.

[illegible]

Please go on to the next page...

Question 11**6 marks**

Fill the table to show the dependencies between the instructions in the following code

```

I1    addi t0, t0, 2
I2    add  t1, t1, t0
I3    mul  t2, t2, t3
I4    bne  t3, t1, label
I5    slli t1, t5, 2
I6    sub  t4, t4, t2
      label:
I7    add  t3, t3, t0
I8    mul  t4, t7, t3
I9    addi t0, t1, 5

```

Use the format T(), A(), O() to represent true, anti, and output dependencies, respectively, with register name in parenthesis. The table already shows one example of a dependency. Assume that the branch instruction is not taken and note that the branch itself does not have an output.

(In case this gets messy, there is an additional copy of the table in Page 20. Please indicate clearly which copy contains your answer.)

	I1	I2	I3	I4	I5	I6	I7	I8	I9
I1		T(t0)							
I2									
I3									
I4									
I5									
I6									
I7									
I8									
I9									

Please go on to the next page...

Question 12**6 marks**

Which of the following is a valid re-order of the code sequence in Question 11 assuming register renaming:

i. I4, I1, I6, I2, I5, I9, I7, I8, I3

☐

ii. I1, I2, I4, I9, I7, I8, I5, I3, I6

☐

iii. I1, I7, I2, I4, I6, I5, I3, I9, I8

☐

iv. I1, I7, I3, I2, I5, I9, I8, I6, I4

☐

Code reproduced here for your convenience:

```
I1    addi t0, t0, 2
I2    add  t1, t1, t0
I3    mul  t2, t2, t3
I4    bne  t3, t1, label
I5    slli t1, t5, 2
I6    sub  t4, t4, t2
      label:
I7    add  t3, t3, t0
I8    mul  t4, t7, t3
I9    addi t0, t1, 5
```

Question 13**6 marks**

Execute the code from Question 11 in a 5-stage pipeline without forwarding and branch-prediction. How many NOPs are needed assuming the branch is not taken?

--



Please go on to the next page...

Question 14**6 marks**

In a fully associative cache with four cache lines, we perform the following sequence of accesses

6 4 3 4 3 1 2 4 5 2 3 4 1 2 3 6 3 4 5 1

Assuming initially none of the addresses is in the cache, determine how many replacements are required for the following replacement policies:

Clairvoyant:

FIFO:

LRU:

For the clairvoyant policy, If multiple addresses do not appear again in the future, choose the leftmost line to replace its content

Please go on to the next page...

Question 15**6 marks**

Given is a multiprocessor system with two processors and one shared memory address A . Each processor has its own cache. We are to implement MESI protocol. Initially, neither processor holds a valid copy of the data from memory address A . Fill the table with MESI states for corresponding cache lines after each operation.

Access	Processor 1	Processor 2
Initially		
Processor 2 reads		
Processor 2 writes		
Processor 1 reads		
Processor 2 reads		
Processor 1 writes		

Please go on to the next page...

Question 16**6 marks**

(a) What is the difference between arithmetic and logical shift?

(b) Write a RISC-V program that takes a signed integer in register `t0` and uses shift operations to divide it by 8. The result should be in register `t1`

Please go on to the next page...

Question 17**5 marks**

Write a RISC-V program that takes an unsigned integer n in register `t0` as input. The program outputs a number in register `t1`. The output is 0 if n is a multiple of 7. Otherwise the output is n .

For example, if the input is 17, the output is 17, but if the input is 14, the output is 0.

Please go on to the next page...

Question 18**6 marks**

Write a RISC-V program that takes an unsigned integer n in register `t0` as input. The program outputs a number in register `t1`. The output is 0 if the units digit in the decimal representation of n is 7. Otherwise the output is n .

For example, if the input is 17, the output is 0, but if the input is 14, the output is 14.

Please go on to the next page...

Question 19**6 marks**

Write a RISC-V program that takes an unsigned integer n in register t0 as input. The program outputs a number in register t1 . The output is 0 if **any** of the digits in the decimal representation of n is 7. Otherwise the output is n .

For example, if the input is 17 or 1701, the output is 0, but if the input is 14, the output is 14.

Please go on to the next page...

Question 20**5 marks**

Write a RISC-V program that takes an unsigned integer n in register t0 as input. The program outputs a number in register t1 . The output is 0 if n is a multiple of 7 or any of the digits in its decimal representation is 7. Otherwise the output is n .

For example, if the input is 77, 17, 1701 or 14, the output is 0, but if the input is 15, the output is 15.

Please go on to the next page...

Base Integer Instructions: RV32I, RV64I, and RV128I						RV Privileged Instructions								
Category		Name	Fmt	RV32I Base		+RV{64,128}		Category		Name	RV mnemonic			
Loads	Load Byte	I	LB	rd,rs1,imm	L{D Q} rd,rs1,imm L{W D}U rd,rs1,imm		CSR Access	Atomic R/W	CSRRW	rd,csr,rs1				
	Load Halfword	I	LH	rd,rs1,imm				Atomic Read & Set Bit	CSRRS	rd,csr,rs1				
	Load Word	I	LW	rd,rs1,imm				Atomic Read & Clear Bit	CSRRC	rd,csr,rs1				
	Load Byte Unsigned	I	LBU	rd,rs1,imm				Atomic R/W Imm	CSRRWI	rd,csr,imm				
	Load Half Unsigned	I	LHU	rd,rs1,imm				Atomic Read & Set Bit Imm	CSRRSI	rd,csr,imm				
Stores	Store Byte	S	SB	rs1,rs2,imm	S{D Q} rs1,rs2,imm		Change Level	Env. Call	ECALL					
	Store Halfword	S	SH	rs1,rs2,imm				Environment Breakpoint	EBREAK					
	Store Word	S	SW	rs1,rs2,imm				Environment Return	ERET					
Shifts	Shift Left	R	SLL	rd,rs1,rs2	SLL{W D} rd,rs1,rs2	Trap Redirect to Supervisor	Redirect Trap to Hypervisor	MRTS						
	Shift Left Immediate	I	SLLI	rd,rs1,shamt	SLLI{W D} rd,rs1,shamt		Hypervisor Trap to Supervisor	MRTH						
	Shift Right	R	SRL	rd,rs1,rs2	SRL{W D} rd,rs1,rs2		Interrupt Wait for Interrupt	WFI						
	Shift Right Immediate	I	SRLI	rd,rs1,shamt	SRLI{W D} rd,rs1,shamt			MMU Supervisor FENCE	SFENCE.VM rs1					
	Shift Right Arithmetic	R	SRA	rd,rs1,rs2	SRA{W D} rd,rs1,rs2									
	Shift Right Arith Imm	I	SRAI	rd,rs1,shamt	SRAI{W D} rd,rs1,shamt									
Arithmetic	ADD	R	ADD	rd,rs1,rs2	ADD{W D} rd,rs1,rs2									
	ADD Immediate	I	ADDI	rd,rs1,imm	ADDI{W D} rd,rs1,imm									
	SUBtract	R	SUB	rd,rs1,rs2	SUB{W D} rd,rs1,rs2									
	Load Upper Imm	U	LUI	rd,imm	Optional Compressed (16-bit) Instruction Extension: RVC									
	Add Upper Imm to PC	U	AUIPC	rd,imm										
Logical	XOR	R	XOR	rd,rs1,rs2	Loads	Load Word	CL	C.LW	rd',rs1',imm	LW	rd',rs1',imm*4			
	XOR Immediate	I	XORI	rd,rs1,imm		Load Word SP	CI	C.LWSP	rd,imm	LW	rd,sp,imm*4			
	OR	R	OR	rd,rs1,rs2		Load Double	CL	C.LD	rd',rs1',imm	LD	rd',rs1',imm*8			
	OR Immediate	I	ORI	rd,rs1,imm		Load Double SP	CI	C.LDSP	rd,imm	LD	rd,sp,imm*8			
	AND	R	AND	rd,rs1,rs2		Load Quad	CL	C.LQ	rd',rs1',imm	LQ	rd',rs1',imm*16			
	AND Immediate	I	ANDI	rd,rs1,imm		Load Quad SP	CI	C.LQSP	rd,imm	LQ	rd,sp,imm*16			
	Compare	Set <	R	SLT		rd,rs1,rs2	Stores	Store Word	CS	C.SW	rs1',rs2',imm	SW	rs1',rs2',imm*4	
Set < Immediate		I	SLTI	rd,rs1,imm	Store Word SP	CSS		C.SWSP	rs2,imm	SW	rs2,sp,imm*4			
Set < Unsigned		R	SLTU	rd,rs1,rs2	Store Double	CS		C.SD	rs1',rs2',imm	SD	rs1',rs2',imm*8			
Set < Imm Unsigned		I	SLTIU	rd,rs1,imm	Store Double SP	CSS		C.SDSP	rs2,imm	SD	rs2,sp,imm*8			
Branches		Branch =	SB	BEQ	rs1,rs2,imm	Store Quad		CS	C.SQ	rs1',rs2',imm	SQ	rs1',rs2',imm*16		
	Branch ≠	SB	BNE	rs1,rs2,imm	Store Quad SP	CSS	C.SQSP	rs2,imm	SQ	rs2,sp,imm*16				
	Branch ≥	SB	BLT	rs1,rs2,imm	Arithmetic	ADD	CR	C.ADD	rd,rs1	ADD	rd,rd,rs1			
	Branch >	SB	BGE	rs1,rs2,imm		ADD Word	CR	C.ADDW	rd,rs1	ADDW	rd,rd,imm			
	Branch < Unsigned	SB	BLTU	rs1,rs2,imm		ADD Immediate	CI	C.ADDI	rd,imm	ADDI	rd,rd,imm			
	Branch ≥ Unsigned	SB	BGEU	rs1,rs2,imm		ADD Word Imm	CI	C.ADDIW	rd,imm	ADDIW	rd,rd,imm			
Jump & Link	J&L	UJ	JAL	rd,imm		ADD SP Imm * 16	CI	C.ADDI16SP	x0,imm	ADDI	sp,sp,imm*16			
	Jump & Link Register	UJ	JALR	rd,rs1,imm		ADD SP Imm * 4	CIW	C.ADDI4SPN	rd',imm	ADDI	rd',sp,imm*4			
	Synch	Synch thread	I	FENCE		Load Immediate	CI	C.LI	rd,imm	ADDI	rd,x0,imm			
Synch Instr & Data		I	FENCE.I		Load Upper Imm	CI	C.LUI	rd,imm	LUI	rd,imm				
System	System CALL	I	SCALL		MoVe	CR	C.MV	rd,rs1	ADD	rd,rs1,x0				
	System BREAK	I	SBREAK		SUB	CR	C.SUB	rd,rs1	SUB	rd,rd,rs1				
Counters	ReaD CYCLE	I	RDCYCLE	rd	Shifts Shift Left Imm	CI	C.SLLI	rd,imm	SLLI	rd,rd,imm				
	ReaD CYCLE upper Half	I	RDCYCLEH	rd	Branches Branch=0	CB	C.BEQZ	rs1',imm	BEQ	rs1',x0,imm				
	ReaD TIME	I	RDTIME	rd		Branch≠0	CB	C.BNEZ	rs1',imm	BNE	rs1',x0,imm			
	ReaD TIME upper Half	I	RDTIMEH	rd	Jump	Jump	CJ	C.J	imm	JAL	x0,imm			
	ReaD INSTR RETired	I	RDINSTRET	rd		Jump Register	CR	C.JR	rd,rs1	JALR	x0,rs1,0			
	ReaD INSTR upper Half	I	RDINSTRETH	rd	Jump & Link J&L	CJ	C.JAL	imm	JAL	ra,imm				
						Jump & Link Register	CR	C.JALR	rs1	JALR	ra,rs1,0			
					System Env. BREAK	CI	C.EBREAK		EBREAK					

32-bit Instruction Formats

	31	30	25	24	21	20	19	15	14	12	11	8	7	6	0		
R	funct7				rs2		rs1	funct3				rd		opcode			
I	imm[11:0]						rs1	funct3				rd		opcode			
S	imm[11:5]				rs2		rs1	funct3		imm[4:0]		opcode					
SB	imm[12]	imm[10:5]				rs2		rs1	funct3		imm[4:1]	imm[11]	opcode				
U	imm[31:12]													rd		opcode	
UJ	imm[20]	imm[10:1]				imm[11]	imm[19:12]						rd		opcode		

16-bit (RVC) Instruction Formats

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CR	funct4				rd/rs1				rs2				op			
CI	funct3		imm		rd/rs1				imm				op			
CSS	funct3		imm						rs2				op			
CIW	funct3		imm								rd'		op			
CL	funct3		imm			rs1'			imm		rd'		op			
CS	funct3		imm			rs1'			imm		rs2'		op			
CB	funct3		offset			rs1'			offset						op	
CJ	funct3		jump target												op	

RISC-V Integer Base (RV32I/64I/128I), privileged, and optional compressed extension (RVC). Registers x1-x31 and the pc are 32 bits wide in RV32I, 64 in RV64I, and 128 in RV128I (x0=0). RV64I/128I add 10 instructions for the wider formats. The RVI base of <50 classic integer RISC instructions is required. Every 16-bit RVC instruction matches an existing 32-bit RVI instruction. See risc.org.

Optional Multiply-Divide Instruction Extension: RVM					
Category	Name	Fmt	RV32M (Multiply-Divide)	+RV{64,128}	
Multiply	MULTiply	R	MUL rd,rs1,rs2	MUL{W D}	rd,rs1,rs2
	MULTiply upper Half	R	MULH rd,rs1,rs2		
	MULTiply Half Sign/Uns	R	MULHSU rd,rs1,rs2		
	MULTiply upper Half Uns	R	MULHU rd,rs1,rs2		
	DIVide	R	DIV rd,rs1,rs2	DIV{W D}	rd,rs1,rs2
Divide	DIVide Unsigned	R	DIVU rd,rs1,rs2		
	REMAinder	R	REM rd,rs1,rs2	REM{W D}	rd,rs1,rs2
Remainder	REMAinder Unsigned	R	REMU rd,rs1,rs2	REMU{W D}	rd,rs1,rs2
Optional Atomic Instruction Extension: RVA					
Category	Name	Fmt	RV32A (Atomic)	+RV{64,128}	
Load	Load Reserved	R	LR.W rd,rs1	LR.{D Q}	rd,rs1
Store	Store Conditional	R	SC.W rd,rs1,rs2	SC.{D Q}	rd,rs1,rs2
Swap	SWAP	R	AMOSWAP.W rd,rs1,rs2	AMOSWAP.{D Q}	rd,rs1,rs2
Add	ADD	R	AMOADD.W rd,rs1,rs2	AMOADD.{D Q}	rd,rs1,rs2
Logical	XOR	R	AMOXOR.W rd,rs1,rs2	AMOXOR.{D Q}	rd,rs1,rs2
	AND	R	AMOAND.W rd,rs1,rs2	AMOAND.{D Q}	rd,rs1,rs2
	OR	R	AMOOR.W rd,rs1,rs2	AMOOR.{D Q}	rd,rs1,rs2
Min/Max	MINimum	R	AMOMIN.W rd,rs1,rs2	AMOMIN.{D Q}	rd,rs1,rs2
	MAXimum	R	AMOMAX.W rd,rs1,rs2	AMOMAX.{D Q}	rd,rs1,rs2
	MINimum Unsigned	R	AMOMINU.W rd,rs1,rs2	AMOMINU.{D Q}	rd,rs1,rs2
	MAXimum Unsigned	R	AMOMAXU.W rd,rs1,rs2	AMOMAXU.{D Q}	rd,rs1,rs2
Three Optional Floating-Point Instruction Extensions: RVF, RVD, & RVQ					
Category	Name	Fmt	RV32{F D Q} (HP/SP,DP,QP FI Pt)	+RV{64,128}	
Move	Move from Integer	R	FMV.{H S}.X rd,rs1	FMV.{D Q}.X	rd,rs1
	Move to Integer	R	FMV.X.{H S} rd,rs1	FMV.X.{D Q}	rd,rs1
Convert	Convert from Int	R	FCVT.{H S D Q}.W rd,rs1	FCVT.{H S D Q}.L{T}	rd,rs1
	Convert from Int Unsigned	R	FCVT.{H S D Q}.WU rd,rs1	FCVT.{H S D Q}.L{T}U	rd,rs1
	Convert to Int	R	FCVT.W.{H S D Q} rd,rs1	FCVT.L{T}.H{S D Q}	rd,rs1
	Convert to Int Unsigned	R	FCVT.WU.H{S D Q} rd,rs1	FCVT.L{T}U.H{S D Q}	rd,rs1
Load	Load	I	FL{W,D,Q} rd,rs1,imm	RISC-V Calling Convention	
Store	Store	S	FS{W,D,Q} rs1,rs2,imm	Register	ABI Name
Arithmetic	ADD	R	FADD.{S D Q} rd,rs1,rs2	x0	zero
	SUBtract	R	FSUB.{S D Q} rd,rs1,rs2	x1	ra
	MULTiply	R	FMUL.{S D Q} rd,rs1,rs2	x2	sp
	DIVide	R	FDIV.{S D Q} rd,rs1,rs2	x3	gp
	SQuare RooT	R	FSQRT.{S D Q} rd,rs1	x4	tp
Mul-Add	MULTiply-ADD	R	FMADD.{S D Q} rd,rs1,rs2,rs3	x5-7	t0-2
	MULTiply-SUBtract	R	FMSUB.{S D Q} rd,rs1,rs2,rs3	x8	s0/fp
	Negative MULTiply-SUBtract	R	FNMSUB.{S D Q} rd,rs1,rs2,rs3	x9	s1
	Negative MULTiply-ADD	R	FNMADD.{S D Q} rd,rs1,rs2,rs3	x10-11	a0-1
Sign Inject	SIGN source	R	FSGNJ.{S D Q} rd,rs1,rs2	x12-17	a2-7
	Negative SIGN source	R	FSGNJN.{S D Q} rd,rs1,rs2	x18-27	s2-11
	Xor SIGN source	R	FSGNJX.{S D Q} rd,rs1,rs2	x28-31	t3-t6
Min/Max	MINimum	R	FMIN.{S D Q} rd,rs1,rs2	f0-7	ft0-7
	MAXimum	R	FMAX.{S D Q} rd,rs1,rs2	f8-9	fs0-1
Compare	Compare Float =	R	FEQ.{S D Q} rd,rs1,rs2	f10-11	fa0-1
	Compare Float <	R	FLT.{S D Q} rd,rs1,rs2	f12-17	fa2-7
	Compare Float ≤	R	FLE.{S D Q} rd,rs1,rs2	f18-27	fs2-11
Categorization	Classify Type	R	FCLASS.{S D Q} rd,rs1	f28-31	ft8-11
Configuration	Read Status	R	FRCSR rd		
	Read Rounding Mode	R	FRRM rd		
	Read Flags	R	FRFLAGS rd		
	Swap Status Reg	R	FSCSR rd,rs1		
	Swap Rounding Mode	R	FSRM rd,rs1		
	Swap Flags	R	FSFLAGS rd,rs1		
	Swap Rounding Mode Imm	I	FSRMI rd,imm		
	Swap Flags Imm	I	FSFLAGSI rd,imm		
				Description	
				---	Hard-wired zero
				Caller	Return address
				Callee	Stack pointer
				---	Global pointer
				---	Thread pointer
				Caller	Temporaries
				Callee	Saved register/frame pointer
				Callee	Saved register
				Caller	Function arguments/return values
				Caller	Function arguments
				Callee	Saved registers
				Caller	Temporaries
				Caller	FP temporaries
				Callee	FP saved registers
				Caller	FP arguments/return values
				Caller	FP arguments
				Callee	FP saved registers
				Caller	FP temporaries

RISC-V calling convention and five optional extensions: 10 multiply-divide instructions (RV32M); 11 optional atomic instructions (RV32A); and 25 floating-point instructions each for single-, double-, and quadruple-precision (RV32F, RV32D, RV32Q). The latter add registers f0-f31, whose width matches the widest precision, and a floating-point control and status register fcsr. Each larger address adds some instructions: 4 for RVM, 11 for RVA, and 6 each for RVF/D/Q. Using regex notation, { } means set, so L{D|Q} is both LD and LQ. See riscv.org. (8/21/15 revision)

Extra table for Question 11

Please indicate clearly which copy contains your answer.

	I1	I2	I3	I4	I5	I6	I7	I8	I9
I1		T(t_0)							
I2									
I3									
I4									
I5									
I6									
I7									
I8									
I9									

Please go on to the next page...

Extra Space for Answers

Please indicate the question number clearly

This image shows a full page of blank graph paper. The grid consists of thin, light gray horizontal and vertical lines that intersect to form small squares across the entire surface. There are no margins, text, or other markings on the paper.

Please go on to the next page...

Extra Space for Answers

Please indicate the question number clearly

Please go on to the next page...

Extra Space for Answers

Please indicate the question number clearly

End of exam